



DURBAN UNIVERSITY OF TECHNOLOGY
INYUVESI YASETHEKWINI YEZOBUCHWEPHESHE

FACULTY OF ACCOUNTING AND INFORMATICS

DEPARTMENT OF INFORMATION TECHNOLOGY

2025

MID YEAR SUPPLEMENTARY EXAMINATION

MARKING MEMORANDUM

INSTRUCTIONAL PROGRAMME : B INFORMATION & COMMUNICATION TECHNOLOGY

INSTRUCTIONAL OFFERING : INTEGRATIVE PROGRAMMING AND TECHNOLOGIES III

PAPER NUMBER : 1

SUBJECT CODE/S : IPRT301

DATE :

DURATION : 3 HOURS

TIME :

TOTAL MARKS : 100

NUMBER OF PAGES : 13 (INCLUDING COVER PAGE)

EXAMINER/S : BOAMAH-ABU, CHARLES (PhD)

MODERATOR/S : RUSHIL RAGHAVJEE (PhD)

INSTRUCTIONS/REQUIREMENTS:-

Do not turn the page until permission is given

Question 1

10 marks

In each of the following, state whether the statement is **true** or **false**. Write **T** (for true) or **F** (for false).

1. Independent, standalone classes promote code reuse.
2. Each instance of a class has its own value of a static variable.
3. Abstract classes cannot contain method declarations.
4. A class can inherit an interface
5. An interface can contain instance variables
6. In distributed applications, each application has processing capability and can perform a specific function.
7. Distributed architectures are design patterns.
8. Low coupling in distributed applications means strong interdependency among the applications.
9. In socket programming, the input stream of the server connects to the output stream of the client.
10. RMI is a broker architecture.

Answer

1. T 2. F 3. F 4. F 5. F 6. T 7. T 8. F 9. T 10. T

Question 2

20 marks

1. **Briefly** answer the following.

1.1 A class **Student** inherits another class **Person**. Use this to explain the concept of polymorphism. (2)

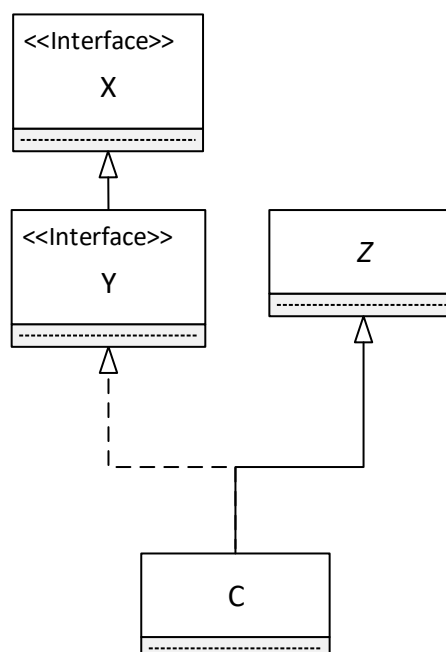
Answer: Any correct answer: Polymorphism means multiple existence: an instance of a subclass can also exist as an instance of the superclass. In this case, an instance of **Student** can exist and behave as both a **Student** and also as a **Person**. ✓✓

1.2 The code below illustrates **method overload**. Explain why this is wrong. (2)

```
package q2_1;
public abstract class A {
    public abstract int a();
    public abstract double a();
}
```

Answer: Any correct answer: method overload is multiple existence of the same method in a class. This means the same method name but different parameter list. In this case, the parameter list is the same. ✓✓✓

2. The UML diagram below shows **X** and **Y** are interfaces, **Y** is an abstract class and **C** is a concrete class.



2.1 Write the header of **Y**. (2)

2.2 Write the header of **Z**. (1)

2.3 Write the header of C.

(3)

Answers:

```
2.1 public interface Y extends X ✓  
2.2 public abstract class Z ✓  
2.3 public class C extends Z implements Y ✓
```

3. In each of the following, indicate whether the statement is **valid** or **invalid**. Briefly explain your answer.

3.1 Y obj1=new Y(); (1½)

3.2 Z obj2=new Z(); (1½)

3.3 Y obj3=new C(); (2)

3.4 Z obj4=new C(); (2)

3.5 X obj5=new C(); (3)

Answers:

```
3.1 Y obj1=new Y(); Invalid; ✓½ cannot instantiate an interface ✓1  
3.2 Z obj2=new Z(); Invalid; ✓½ cannot instantiate an abstract class ✓1½  
3.3 Y obj3=new C(); Valid; ✓½ C is concrete and implements Y ✓1½  
3.4 Z obj4=new C(); Valid; ✓½ C is a concrete class and inherits Z ✓1½  
3.5 X obj5=new C(); Valid, ✓½ C is a concrete class; ✓½ Y inherits X and, ✓1 therefore, C  
"inherits" X through Y ✓1
```

Question 3

20 marks

1. What are following design patterns used for? (4)

1.1. Factory

1.2. Template

1.2.1 The factory pattern is used to create instances of subclasses as instances of the superclass

1.2.2 The template pattern is used to specify the methods in interfaces and abstract superclasses and how these methods can be implemented by implementation methods or subclasses.

2. The class **PhoneBook** (below) is a **Singleton** class based on **lazy initialization**.

```
PhoneBook
  S pb : PhoneBook
  A name : String
  C PhoneBook(String)
  S createInstance(String) : PhoneBook
  A toString() : String

Tester
  S main(String[]) : void
```

- **pb** is a variable of type **PhoneBook**
- **name** is a string (name of a phone book)
- **PhoneBook** is a constructor, it takes a string (the **name** of a phone book) as a parameter
- the method **createInstance** uses **lazy initialization** to create and return an instance of **PhoneBook**
- **toString** returns a string representation of an instance of **PhoneBook**

2.1 Complete the class **PhoneBook** (write only the missing part). (10)

```
package q3;
```

```
public class PhoneBook {
```

```
_____  
_____  
_____  
_____  
_____  
_____  
_____  
_____  
_____
```

```

    public String toString() {
        return "Phone book: " + name;
    }
}

```

Answer

```

package q3;

public class PhoneBook {
    private static PhoneBook pb; ✓1½
    String name; ✓
    private PhoneBook(String namePar) { ✓1½
        name=namePar;
    }
    public static ✓ PhoneBook createInstance(String namePar) { ✓✓
        if(pb==null) { ✓
            pb=new PhoneBook(namePar); ✓
        }
        return pb; ✓
    }
    public String toString() {
        return "Phone book: " + name;
    }
}

```

2.2 The **main** method in the **Tester** creates an instance of **PhoneBook** (called **p**) with a name "Joe's" (as highlighted below).

```

public class Tester {
    public static void main(String[] args) {
        PhoneBook p= new PhoneBook("Joe's");
        System.out.println(p);
    }
}

```

2.2.1 What is wrong with the line of code? (2)

Answer: The constructor in singleton (lazy initialization) is private and, therefore, cannot be called ✓✓

2.2.2 Write the correct code.

(2)

```
public class Tester {  
    public static void main(String[] args) {  
        PhoneBook p= PhoneBook.createInstance("Joe's");✓✓  
        System.out.println(p);  
    }  
}
```

2.3 Modify the header of **createInstance** so that the method is thread-safe (i.e. allows only one thread to enter at a time).

(2)

Answer: **public synchronized**✓ **static** PhoneBook createInstance(String namePar)✓

Question 4

25 marks

1. **Briefly** answer the following.

1.1 What is the main benefit of XML in Integrative programming? (2)

1.2 Why is an XML schema important? (2)

Any correct answer: e.g., XML is a data exchange standard and allows different applications to share data ✓✓

Any correct answer: e.g., specifies the structure of the XML in an XML document ✓✓

2. LFA Restaurant has many part-time employees. Below is part of the data on employees: it shows the **id** and **name** of employees and the **number of hours** the employees worked.

01	Joe Hart	10
02	Ann May	20
03	Dan Pele	10

The data is stored in an XML file called **wage.xml**. The root node is **Wage**, which contains many **Employee** nodes. Each **Employee** node contains the elements **eid**, **name** and **num** (i.e., employee **eid**, **name** and **number of hours worked**).

2.1 Complete the XML code (below) for the three employees in the file **wage.xml**. (7)

```
<?xml version="1.0" encoding="UTF-8"?>
```

Answer

```
<Wage>✓  
  <Employee>✓  
    <eid>01</eid>✓  
    <name>Joe Hart</name>✓  
    <num>10</num>✓  
  </Employee>  
  <Employee>✓  
    <eid>02</eid>  
    <name>Ann May</name>  
    <num>20</num>
```

```

    </Employee>
    <Employee>✓
        <eid>03</eid>
        <name>Dan Pele</name>
        <num>10</num>
    </Employee>
</Wage>

```

2.2 Complete the **DTD** code (below) in a file **wage.dtd** for the XML in **wage.xml**. (6)

```

<?xml version="1.0" encoding="UTF-8"?>

```

Answer

```

<?xml version="1.0" encoding="UTF-8"?>
<!ELEMENT Wage (Employee)> ✓✓
<!ELEMENT Employee (id, name, num)> ✓✓
<!ELEMENT id (#PCDATA)> ✓
<!ELEMENT name (#PCDATA)> ✓
<!ELEMENT num (#PCDATA)>

```

2.3 The data in **wage.xml** is to be displayed (as shown below) in a web page.

Wages (@ R100/hour)

Staff Id	Name	Hours	Wages
01	Joe Hart	10	1000
02	Ann May	20	2000
03	Dan Pele	10	1000

The XSL is in a file **wage.xsl**. Wages are calculated at R100 per hour. Complete the missing parts of the XSL code (below). (7)

```

<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
    <xsl:template match="/">
        <title>Exams Question 4: Salaries in a Department</title>
        <h3>Wages (@ R100/hour)</h3>
        <hr width="50%" align="left"/>
        <table border="1">
            <tr>
                <th>Staff Id</th>
                <th>Name</th>

```

```

        <th>Hours</th>
    </tr>

    <xsl:for-each select="_____">
        <tr>
            <td> <_____ /> </td>
            <td> <_____ /> </td>
            <td> <_____ /> </td>
            <td> <_____ /> </td>

        </tr>
    </xsl:for-each>
</table>
</xsl:template>
</xsl:stylesheet>

```

Answer

```

<xsl:for-each select="Wage/EmpLoyee">✓✓
    <tr>
        <td><xsl:value-of select="eid"✓✓ /> </td>
        <td><xsl:value-of select="name"✓ /></td>
        <td><xsl:value-of select="num" ✓/></td>
        <td><xsl:value-of select="num*100"✓ /></td>

    </tr>
</xsl:for-each>

```

2.4 Complete the line of code below to link **wage.xml** to the stylesheet in **wage.xsl**. (1)

```
<?xml-stylesheet type="text/xsl" _____?>
```

Answer

```
<?xml-stylesheet type="text/xsl" href="wage.xsl"✓?>
```

Question 5

25 marks

1. An application consists of a server and a client. **Briefly** explain what will happen if the client executes before the server. (2)

Answer: the application will throw an exception because the client will try to establish a connection to the server.

2. Sharon Hotel offers accommodation to tourists. Each room is R1000 per night. However, if a booking is for 10 or more rooms and for two or more days, there is a discount of 10% on the total cost of rooms. You are required to write an RMI application as follows.

IBooking is a remote interface; the class **IBookingImp** implements the interface; **IBooking** and **IBookingImp** use the class **Booking** (see the figure below).

```
classDiagram
    class Booking {
        id : String
        num : int
        duration : int
        Booking()
        Booking(String, int, int)
        getId() : String
        setId(String) : void
        getNum() : int
        setNum(int) : void
        getDuration() : int
        setDuration(int) : void
        toString() : String
    }
    class IBooking {
        ROOMPRICE : double
        DISCPERCENT : double
        calcTotalRoomCost(Booking) : double
        calcDiscount(Booking) : double
        calcAmountDue(Booking) : double
    }
    class IBookingImp {
        IBookingImp()
        calcTotalRoomCost(Booking) : double
        calcDiscount(Booking) : double
        calcAmountDue(Booking) : double
    }
    IBooking <|-- IBookingImp
    Booking "1" -- "1" IBooking
    Booking "1" -- "1" IBookingImp
```

The diagram shows three classes: **Booking**, **IBooking**, and **IBookingImp**. **Booking** is a concrete class with attributes `id : String`, `num : int`, and `duration : int`. It has constructors `Booking()` and `Booking(String, int, int)`, and methods `getId() : String`, `setId(String) : void`, `getNum() : int`, `setNum(int) : void`, `getDuration() : int`, `setDuration(int) : void`, and `toString() : String`. **IBooking** is a remote interface with constants `ROOMPRICE : double` and `DISCPERCENT : double`, and methods `calcTotalRoomCost(Booking) : double`, `calcDiscount(Booking) : double`, and `calcAmountDue(Booking) : double`. **IBookingImp** is a class that implements **IBooking**, with methods `IBookingImp()`, `calcTotalRoomCost(Booking) : double`, `calcDiscount(Booking) : double`, and `calcAmountDue(Booking) : double`. The diagram also shows associations between **Booking** and both **IBooking** and **IBookingImp**.

- **Booking** has **id** (the booking id), **num** (the number of rooms) and **duration** (number of days), **constructors** and **get**, **set** and **toString** methods.
- **IBooking** has **ROOMPRICE** (the price of a room per night, a constant equal to R1000.00), **DISCPERCENT** (the percentage discount, a constant equal to 10%).
- **calcTotalRoomCost** takes an instance of **Booking** as a parameter and returns the total cost of the rooms booked (calculated as the number of rooms by days).
- **calcDiscount** takes an instance of **Booking** as a parameter and returns the discount (if the number of rooms is 10 or more and the duration is two or more days, the discount is calculated at 10% of the total cost of rooms, otherwise no discount).

- **calcAmountDue** takes an instance of **Booking** as a parameter and returns the amount due (calculated as total cost of rooms less discount).

2.1. Write the code for **IBooking**. (7)

```
public interface IBooking extends Remote{
    double ROOMPRICE=1000;
    double DISCPERCENT=0.1;
    double calcTotalRoomCost(Booking book) throws Exception;
    double calcDiscount(Booking book) throws Exception;
    double calcAmountDue(Booking book) throws Exception;
}
```

2.2. Write the following parts of the code for the class **IBookingImp**.

2.2.1. The class header (3)

2.2.2. The header of the default constructor (2)

2.2.3. The method **calcDiscount** (6)

Answer

```
...
...
public class IBookingImp extends UnicastRemoteObject implements
IBooking {
    public IBookingImp() throws Exception{
    }

    public double calcTotalRoomCost(Booking r) {
        return ROOMPRICE*r.getNum()*r.getDuration();
    }
    public double calcDiscount(Booking r) {
        int extraNum=r.getNum()-10;
        int extraDuration=r.getDuration()-1;
        double disc=0;
        if((extraNum>=0)&&(extraDuration>0)) {
            disc=calcTotalRoomCost(r)*DISCPERCENT;
        }
        return disc;
    }
    public double calcAmountDue(Booking r) {
        return calcTotalRoomCost(r)-calcDiscount(r);
    }
}
```

2.3. Complete **BookingServer** (below) to register and bind an instance (called **ibi**) of **IBookingImp** with the reference **booking**, using port **2025**. Assume all the other classes have been implemented. (5)

```
...
...
public class BookingServer {
    public static void main(String[] args) _____{
        _____
        _____
        _____
    }
}
```

Answer

```
public class BookingServer {
    public static void main(String[] args) throws Exception{
        IBookingImp ibi=new IBookingImp();✓
        Registry reg=LocateRegistry.createRegistry(2025); ✓✓
        reg.rebind("booking", ibi); ✓✓
    }
}
```